Linear Regression

Linear regression adalah metode yang digunakan untuk memahami dan memprediksi hubungan antara satu atau lebih variabel input/independent (fitur) dengan variabel output/dependent (target).

Tujuan Linear Regression:

1. Memahami hubungan antar variabel
   Linear regression digunakan untuk mengetahui hubungan variabel independen atau bebas (x) dengan variabel dependen atau terikat (y) serta bagaimana variabel independen atau bebas (x) dapat mempengaruhi variabel dependen atau terikat (y).
   Contoh : Mengetahui apakah lamanya jam belajar siswa dapat mempengaruhi nilai ujian nya? dan seberapa besar pengaruhnya?
2. Memprediksi nilai baru
   Disini linear regression dapat memprediksi nilai variabel dependen (y) berdasarkan nilai variabel independen (x).
   Contoh : Memperkirakan nilai selanjutnya, jika siswa tersebut belajar sekitar 5 jam per hari.

Berikut **beberapa** tahapan sebelum melakukan pemodelan (Modelling) :

1. Data Preparation

- Import Library

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Daftar diatas adalah library yang umum digunakan untuk analisis dan visualisasi data, berikut kegunaannya :

1. **NumPy:** Pustaka yang menyediakan fungsi-fungsi untuk bekerja dengan array dan operasi matematika.
2. **Pandas:** Pustaka untuk memanipulasi data dalam bentuk tabel (DataFrame), seperti mengimpor, membersihkan, dan menganalisis data.
3. **Seaborn:** Pustaka yang digunakan untuk membuat visualisasi data yang lebih menarik dan informatif, seperti grafik distribusi dan hubungan antar variabel.
4. **Matplotlib:** Pustaka yang berfungsi untuk membuat grafik dan plot secara fleksibel.

- **Load dataset**

```
df = pd.read_csv('visitor_income_dataset.csv')
```

Fungsi **read_csv()** dari library Pandas untuk membaca file CSV yang berisi data dan memuatnya ke dalam sebuah DataFrame.

```
df = pd.read_excel('visitor_income_dataset.csv')
```

Fungsi **read_excel()** dari library Pandas untuk membaca file Excel yang berisi data dan memuatnya ke dalam sebuah DataFrame.

- **Menampilkan Beberapa Data**
  1. **Menampilkan 5 Data Teratas**

```
df.head()
```

	Unnamed: 0	Id	Visitor	Income
0	NaN	16	95	1444.377125
1	NaN	10	54	1045.425600
2	NaN	101	11	1987.846293
3	NaN	133	83	1319.376963
4	NaN	69	25	753.616360

  2. **Menampilkan 5 Data terbawah**

```
df.tail()
```

	Unnamed: 0	Id	Visitor	Income
200	NaN	107	74	1258.861859
201	NaN	15	89	1372.750822
202	NaN	93	85	1342.979469
203	NaN	180	76	1287.201692
204	NaN	103	96	1456.572855

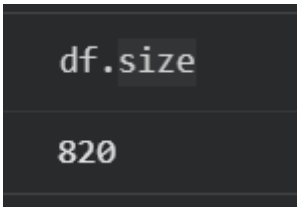
- **Menampilkan Dimensi Data**

```
df.shape
```

```
(205, 4)
```

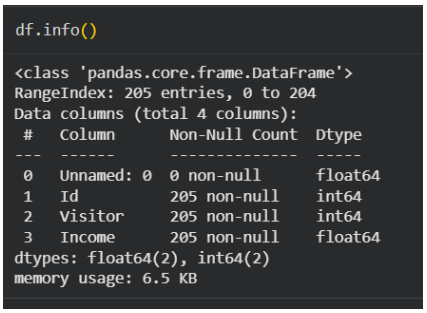
Fungsi **.shape** digunakan untuk mengetahui ukuran atau dimensi dari DataFrame. Pada gambar di atas menunjukkan bahwa DataFrame memiliki 205 baris dan 4 kolom.

- **Menampilkan Jumlah Data**



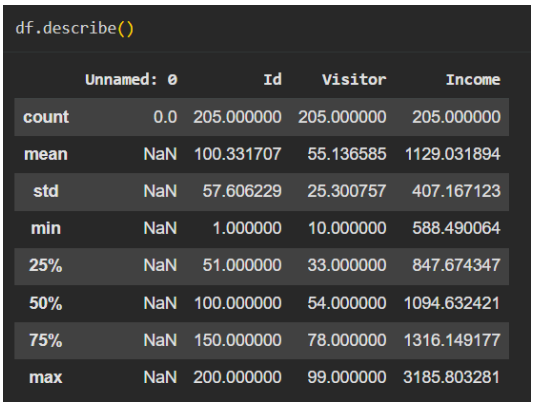
Fungsi **.size** digunakan untuk mengetahui jumlah data dari Dataframe.

- **Menampilkan Informasi Data**



`.info()` adalah fungsi di Pandas yang memberikan informasi ringkas tentang DataFrame. Seperti jumlah baris, jumlah kolom, tipe data setiap kolom, dan jumlah nilai non-null di setiap kolom.

- **Menampilkan Deskripsi Data**



	Unnamed: 0	Id	Visitor	Income
count	0.0	205.000000	205.000000	205.000000
mean	NaN	100.331707	55.136585	1129.031894
std	NaN	57.606229	25.300757	407.167123
min	NaN	1.000000	10.000000	588.490064
25%	NaN	51.000000	33.000000	847.674347
50%	NaN	100.000000	54.000000	1094.632421
75%	NaN	150.000000	78.000000	1316.149177
max	NaN	200.000000	99.000000	3185.803281

`df.describe()` adalah fungsi di Pandas yang memberikan ringkasan statistik dari DataFrame, termasuk nilai seperti rata-rata, standar deviasi, nilai minimum, nilai maksimum, dan kuartil untuk kolom numerik. Fungsi ini berguna untuk memahami distribusi data secara cepat.

- **Memeriksa Tipe Data**
  1. **Menampilkan hanya satu kolom**

```
df["Visitor"].dtype

dtype('int64')
```

2. **Menampilkan setiap kolom**

```
df.dtypes
```

	0
Unnamed: 0	float64
Id	int64
Visitor	int64
Income	float64

- **Menampilkan Nama Kolom**

```
df.columns

Index(['Unnamed: 0', 'Id', 'Visitor', 'Income'], dtype='object')
```

- **Menampilkan Jumlah Tiap Nilai Unik**

```
df['Price_Class'].value_counts()

Price_Class
2    620
1    438
0    245
Name: count, dtype: int64
```

Biasa dilakukan untuk data berlabel seperti pada algoritma kelompok klasifikasi, digunakan untuk mengetahui jumlah data per label sehingga bisa tahu apakah tiap label memiliki data yang balance.

- **Menampilkan Nilai Unik**

```
df["Visitor"].unique()

array([95, 54, 11, 83, 25, 78, 26, 44, 59, 71, 32, 69, 73, 72, 89, 10, 14,
       17, 64, 61, 87, 18, 79, 29, 35, 51, 53, 55, 23, 37, 36, 92, 12, 82,
       84, 52, 86, 70, 60, 88, 47, 13, 33, 49, 77, 24, 67, 68, 50, 30, 96,
       57, 43, 38, 90, 34, 45, 66, 85, 27, 42, 94, 28, 22, 80, 40, 19, 98,
       65, 15, 74, 31, 48, 62, 46, 76, 81, 41, 16, 39, 99, 75])

df["Visitor"].nunique()

82
```

Kode ini menampilkan nilai unik dalam sebuah kolom, fungsi **unique()** digunakan untuk mengembalikan semua nilai unik dalam sebuah kolom dalam bentuk array, tanpa adanya duplikasi. Sementara itu, fungsi **nunique()**

menghitung dan mengembalikan jumlah nilai unik yang terdapat dalam kolom tersebut.

- **Menghapus Kolom Yang Tidak Diperlukan**

```
df = df.drop(columns=["Unnamed: 0"], axis=1)
```

Kode ini berguna untuk menghapus sebuah kolom dari DataFrame menggunakan fungsi **drop().** Parameter **columns=''** menentukan kolom yang akan dihapus, sementara **axis=1** menunjukkan bahwa yang dihapus adalah kolom (bukan baris).

- **Menganalisis Missing Value**
  1. **Menganalisis Missing Value di Setiap Kolom**

```
df.isnull().sum()
```

	0
Id	0
Visitor	0
Income	0

dtype: int64

Fungsi **.isnull().sum()** digunakan untuk mengecek jumlah nilai yang hilang dalam setiap kolom di DataFrame. Fungsi **isnull()** akan menghasilkan nilai **True** untuk setiap data yang hilang, dan kemudian **sum()** akan menghitung jumlah nilai **True** tersebut di setiap kolom.

  2. **Menganalisis Missing Value di Seluruh Dataframe**

```
df.isnull().sum().sum()
```

```
np.int64(0)
```

Fungsi **.isnull().sum().sum()** digunakan untuk menghitung total nilai yang hilang dalam DataFrame dengan cara menghitung jumlahnya di seluruh kolom dan baris.

- **Menangani Missing Value**
  1. **Menghapus Nilai Kosong**

```
df.dropna(inplace=True)
```

Kode ini digunakan untuk menghapus baris yang mengandung nilai hilang dari DataFrame. Dengan menggunakan parameter **inplace=True,** perubahan ini dilakukan langsung pada DataFrame tanpa perlu membuat variabel baru.

2. **Mengganti Dengan Nilai Median**

```
df["Visitor"] = df["Visitor"].fillna(df["Visitor"].median())
```

Kode ini digunakan untuk mengganti nilai yang hilang di kolom Salary dengan nilai tengah dari kolom tersebut. Fungsi **fillna()** akan mencari nilai yang hilang (NaN) dan menggantinya dengan nilai tengah yang dihitung dari data yang ada. Cara ini untuk mengisi data yang hilang tanpa menghapus baris yang mengandung nilai kosong.

3. **Mengganti Dengan Nilai Mean**

```
df["Visitor"] = df["Visitor"].fillna(df["Visitor"].mean())
```

Kode ini digunakan untuk mengganti nilai yang hilang di kolom lalu diganti dengan nilai rata-rata dari kolom tersebut.

4. **Mengganti Dengan Nilai Modus**

```
df["Visitor"] = df["Visitor"].fillna(df["Visitor"].mode()[0])
```

Kode ini digunakan untuk mengganti nilai yang hilang di suatu kolom dengan nilai modus, yaitu nilai yang paling sering muncul dari kolom tersebut. Fungsi **mode()** menghitung nilai modus dalam kolom, dan karena **mode()** menghasilkan lebih dari satu nilai dalam bentuk Series. Agar tidak terjadi seperti itu, gunakan **[0]** untuk memilih nilai pertama.

5. **Mengganti Dengan Nilai Tertentu**

```
df["Visitor"] = df["Visitor"].fillna("kosong")
```

- **Menganalisis Nilai Duplikat**

```
df.duplicated().sum()
```

```
5
```

Kode ini digunakan untuk menghitung jumlah baris duplikat dalam DataFrame. Fungsi duplicated() akan mengembalikan nilai True untuk setiap baris yang merupakan duplikat dari baris sebelumnya dan False untuk baris yang unik. Kemudian, fungsi sum() menghitung jumlah nilai True, yaitu jumlah baris duplikat.

- **Menangani Nilai Duplikat**

```
df = df.drop_duplicates()
```

Kode ini digunakan untuk menghapus baris duplikat dalam DataFrame. Fungsi **drop_duplicates() untuk** memeriksa dan menghapus baris yang memiliki data

yang sama, sementara **inplace=True** untuk melakukan perubahan langsung pada DataFrame tanpa membuat variable baru.

- **Konversi Tipe Data**
  1. **Mengubah Tipe Data Sesama Numerik**

```
df['Embarked'] = df['Embarked'].astype(int)
```

Kode ini mengubah tipe data sebuah kolom menjadi tipe data integer. Fungsi **astype** digunakan untuk mengonversi nilai dalam kolom tersebut ke tipe data lain. Contoh: kolom yang awalnya memiliki tipe data float atau string diubah menjadi integer. String dapat diubah menjadi numerik jika seluruh nilainya berupa angka. Namun, jika ada huruf atau karakter non-numerik, akan muncul error saat mencoba mengonversinya.

  2. **Mengubah Tipe Data String Ke Numerik**

```
df['Embarked'] = pd.factorize(df['Embarked'])[0]
```

pd.factorize() adalah fungsi dari library Pandas yang digunakan untuk mengonversi nilai kategori dalam kolom menjadi angka. Fungsi ini memberikan angka unik untuk setiap kategori.

  3. **Mengubah Tipe Data Kategori Ke Numerik (Mapping)**

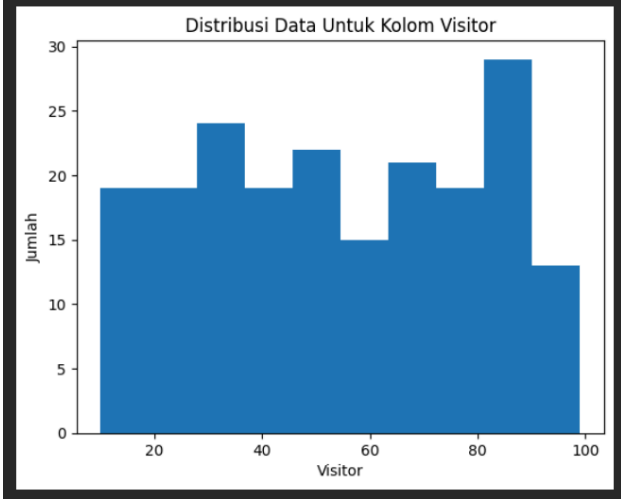
```
df['Embarked'] = df['Embarked'].map({'C': 0, 'Q': 1, 'S': 2})
```

map adalah fungsi di Pandas yang digunakan untuk memetakan nilai di sebuah kolom berdasarkan aturan yang kita buat sendiri.

2. Exploratory Data Analysis

- **Histogram**

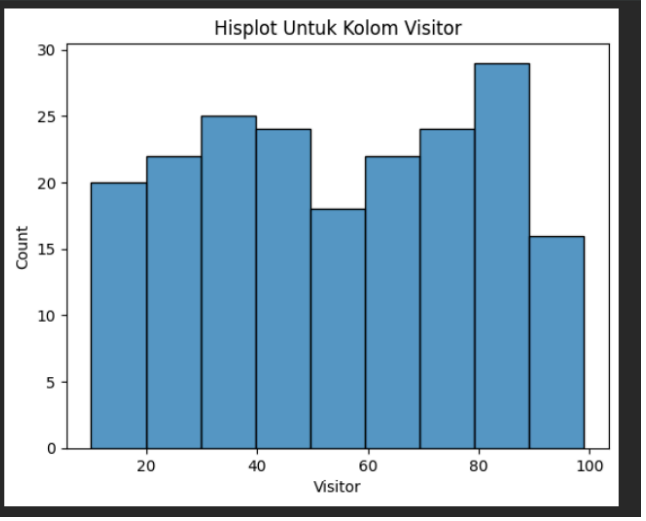
```
plt.hist(df["Visitor"], bins=10)
plt.xlabel("Visitor")
plt.ylabel("Jumlah")
plt.title("Distribusi Data Untuk Kolom Visitor")
plt.show()
```



Histogram ini digunakan untuk memvisualisasikan distribusi data numerik dengan cara membagi data ke dalam sejumlah interval (bin) dan menghitung frekuensi kemunculan data di setiap interval tersebut. Ini membantu memahami pola distribusi, seperti apakah data bersifat normal, miring, atau memiliki outlier. Berasal dari library matplotlib.

- ## Hisplot

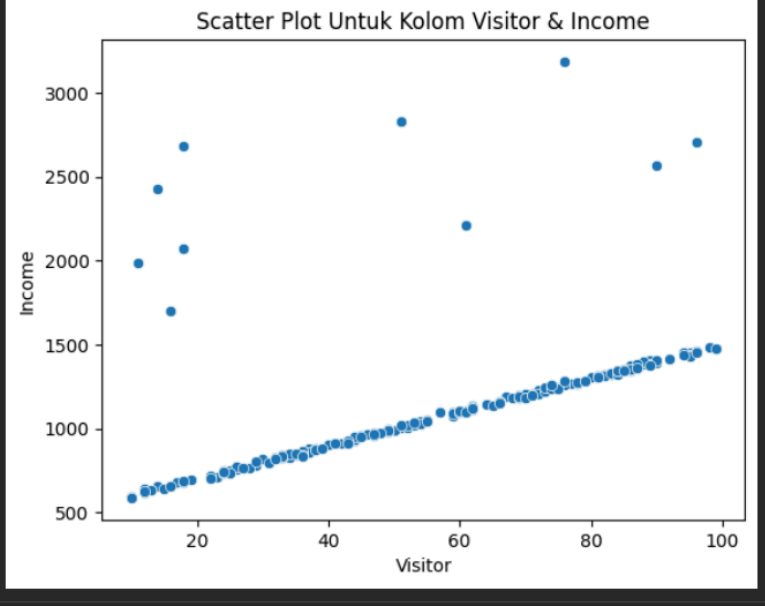
```
sns.histplot(df["Visitor"])
plt.title("Hisplot Untuk Kolom Visitor")
plt.show()
```



Histplot ini digunakan untuk memvisualisasikan distribusi data numerik dengan cara membagi data ke dalam sejumlah interval (bin) dan menghitung frekuensi kemunculan data di setiap interval tersebut. Ini membantu memahami pola distribusi, seperti apakah data bersifat normal, miring, atau memiliki outlier. Berasal dari library seaborn.

- ## Scatter Plot

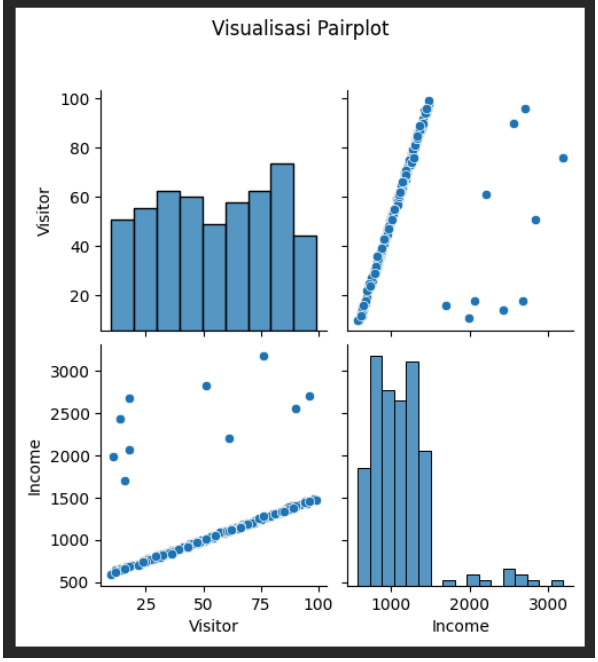
```
sns.scatterplot(data=df, x='Visitor', y='Income')
plt.title("Scatter Plot Untuk Kolom Visitor & Income")
plt.show()
```



Scatter plot digunakan untuk memvisualisasikan hubungan atau pola antara dua variabel numerik. Setiap titik pada scatter plot merepresentasikan satu pasangan data, dengan sumbu horizontal (x) menunjukkan nilai variabel pertama dan sumbu vertikal (y) menunjukkan nilai variabel kedua. Plot ini berguna untuk mengidentifikasi pola, tren, korelasi, atau outlier dalam data.

- **Pairplot**

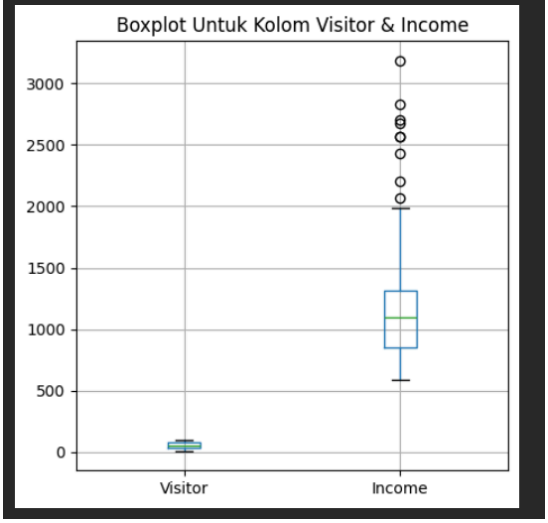
```
g = sns.pairplot(df, vars=['Visitor', 'Income'])
g.fig.suptitle("Visualisasi Pairplot", y=1.1)
plt.show()
```



Pairplot berguna untuk membuat visualisasi hubungan antara pasangan-pasangan variabel dalam sebuah dataset. Fungsi ini menghasilkan plot scatter untuk setiap pasangan variabel numerik dan histogram atau KDE (Kernel Density Estimate) untuk distribusi setiap variabel. Pairplot sangat berguna untuk mengeksplorasi korelasi, pola hubungan, dan distribusi data secara keseluruhan dalam dataset.

- **Menganalisis Outlier Dengan Boxplot**
  1. **Menampilkan Beberapa Kolom**

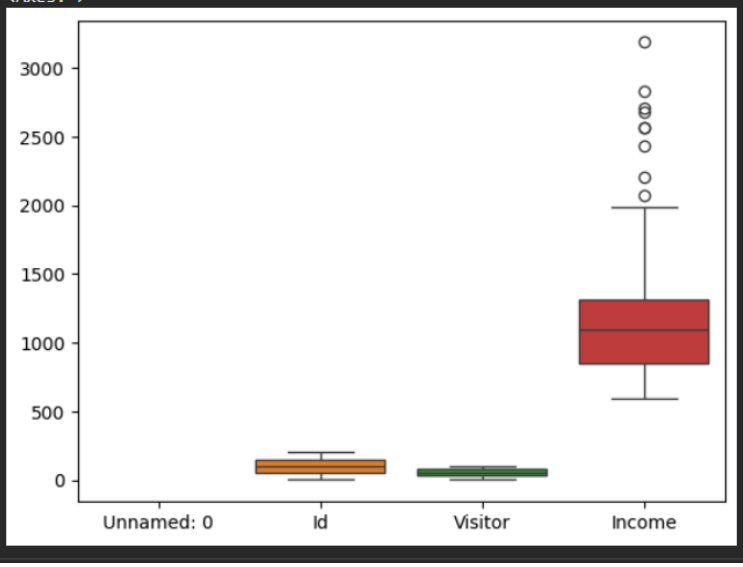
```
plt.figure(figsize=[5,5])
df.boxplot(column=['Visitor', 'Income'])
plt.title("Boxplot Untuk Kolom Visitor & Income")
plt.show()
```



  2. **Menampilkan Seluruh Kolom**

```
sns.boxplot(df)
```

```
<Axes: >
```



- **Menganalisis Outlier Dengan IQR (Interquartile Range)**
  1. **Menghitung jumlah outlier**

```
# Menentukan kuartil atas dan bawah
Q1 = df['Income'].quantile(0.25)
Q3 = df['Income'].quantile(0.75)

# Menghitung Interquartile Range
IQR = Q3 - Q1

# Menghitung batas atas dan bawah
batas_bawah =  Q1 - 1.5 * IQR
batas_atas = Q3 + 1.5 * IQR

# Menghitung jumlah Outlier Pada Kolom Income
outlier = ((df['Income'] < batas_bawah) | (df['Income'] > batas_atas)).sum()
print("Outlier pada kolom Income berjumlah :", outlier)
```

```
Outlier pada kolom Income berjumlah : 8
```

Pertama, kode ini menghitung kuartil pertama (Q1) dan kuartil ketiga (Q3) menggunakan fungsi **quantile(0.25)** untuk Q1 dan **quantile(0.75)** untuk Q3. Selanjutnya, Interquartile Range (IQR) dihitung dengan mengurangi Q1 dari Q3. Kemudian menghitung batas atas dan batas bawah. Nilai-nilai yang berada di luar rentang ini dianggap sebagai outlier.

IQR ini cocok untuk data yang distribusi nya tidak normal dan miring. Visualisasi Boxplot juga menggunakan metode IQR ini untuk menentukan outlier.

2. Menampilkan nilai yang termasuk ke dalam outlier

```
outliers = df[(df['Income'] < batas_bawah) | (df['Income'] > batas_atas)]
outliers
```

	Visitor	Income
20	61	2208.208258
31	51	2827.910319
72	96	2707.274599
82	90	2565.253183
117	18	2069.967142
126	14	2431.384666
148	76	3185.803281
197	18	2679.656488

- Menangani Outlier

1. Mengubah Outlier Dengan Median

```
df.loc[(df['Visitor'] < batas_bawah) | (df['Visitor'] > batas_atas), 'Visitor'] = df['Visitor'].median()
```

Setelah menghitung batas atas dan batas bawah, lalu bisa diubah dari nilai outlier dalam sebuah kolom menjadi nilai median. Fungsi **.loc** untuk mencari nilai yang lebih kecil dari batas bawah atau lebih besar dari batas atas.

2. Mengubah Outlier Dengan Mean

```
df.loc[(df['Visitor'] < batas_bawah) | (df['Visitor'] > batas_atas), 'Visitor'] = df['Visitor'].mean()
```

Setelah menghitung batas atas dan batas bawah, lalu bisa diubah dari nilai outlier dalam sebuah kolom menjadi nilai mean. Fungsi **.loc** untuk mencari nilai yang lebih kecil dari batas bawah atau lebih besar dari batas atas.

3. Mengubah Outlier Dengan Modus

```
df.loc[(df['Visitor'] < batas_bawah) | (df['Visitor'] > batas_atas), 'Visitor'] = df['Visitor'].mode()[0]
```

Setelah menghitung batas atas dan batas bawah, lalu bisa diubah dari nilai outlier dalam sebuah kolom menjadi nilai modus. Fungsi **.loc** untuk mencari nilai yang lebih kecil dari batas bawah atau lebih besar dari batas atas.

4. Menghapus Outlier

```
df = df[~((df['Visitor'] < batas_bawah) | (df['Visitor'] > batas_atas))]
```

Setelah menghitung batas atas dan batas bawah, juga bisa untuk menghapus nilai outlier dalam sebuah kolom. Fungsi **`df[~((df['G2'] < batas_bawah) | (df['G2'] > batas_atas))]`** untuk menyaring DataFrame, hanya menyisakan baris yang nilai **kolom**-nya berada dalam rentang yang valid, yaitu antara batas bawah dan batas atas.

5. Membatasi Nilai Dengan Clip

```
df['Visitor'] = df['Visitor'].clip(0, 500)
```

Selain itu, outlier juga bisa di clip. Clip adalah metode penanganan outlier dengan cara “memotong” nilai ekstrem agar berada dalam batas tertentu. Pada kode tersebut, nilai yang berada dibawah 0 akan dipaksa menjadi nilai 0. Begitu juga dengan nilai yang berada diatas 500 akan dipaksa menjadi nilai 500.

6. Menghapus Outlier Namun Dibatasi

```
df = df[(df['Visitor'] >= 5) & (df['Visitor'] <= 500)]
```

Disini menghapus data namun dibatasi atau di filter, dimana hanya mengambil data kolom visitor yang lebih besar atau sama dengan 5 dan lebih kecil atau sama dengan 500.

- Menganalisis Outlier Dengan Z-score

```python
# Menghitung z-score
z_score = (df['Income'] - df['Income'].mean()) / df['Income'].std()

# Menghitung outlier dengan threshold 3
outlier = (np.abs(z_score) > 3)

# Menampilkan jumlah outlier di kolom Income
print("Jumlah Outlier Di Kolom Income :", outlier.sum())
print("="*35)

# Menampilkan niali yang termasuk outlier
df_outlier = df[outlier]
print(df_outlier)
```

```
Jumlah Outlier Di Kolom Income : 6
==================================
     Visitor       Income
31        51  2827.910319
72        96  2707.274599
82        90  2565.253183
126       14  2431.384666
148       76  3185.803281
197       18  2679.656488
```

Metode analisis dengan z-score cocok untuk data yang distribusinya normal (bell curve) serta data tidak miring.

- **Menangani Outlier Berbasis Z-score**
  1. **Menghapus Outlier**

```python
df_cleaned = df[~outlier]
df_cleaned.head()
```

  2. **Mengubah Outlier Dengan Median**

```python
df.loc[outlier, 'Income'] = df['Income'].median()
```

  3. **Mengubah Outlier Dengan Mean**

```python
df.loc[outlier, 'Income'] = df['Income'].mean()
```

  4. **Mengubah Outlier Dengan Modus**

```python
df.loc[outlier, 'Income'] = df['Income'].mode()[0]
```

- **Menghitung Covariance & Variance Data**

```
print("Berikut covarians data:")
print(df.cov())
print("="*40)
print("Berikut variance data:")
print(df.var())
```

```
Berikut covarians data:
             Visitor         Income
Visitor   643.627513    5603.875813
Income   5603.875813  159214.624709
========================================
Berikut variance data:
Visitor       643.627513
Income     159214.624709
dtype: float64
```

- **Menghitung Korelasi Data**
  1. **Metode Kendall**

```
corr_kendall = df.corr(method='kendall')
corr_kendall
```

	Visitor	Income
Visitor	1.000000	0.866437
Income	0.866437	1.000000

Metode ini digunakan untuk data ordinal (data yang berurutan namun jarak tidak pasti) serta dataset yang cenderung kecil.

  2. **Metode Spearman**

```
corr_spearman = df.corr(method='spearman')
corr_spearman
```

	Visitor	Income
Visitor	1.000000	0.855249
Income	0.855249	1.000000

Digunakan untuk hubungan yang tidak linear namun searah.

  3. **Metode Pearson**

```
corr_pearson = df.corr(method='pearson')
corr_pearson
```

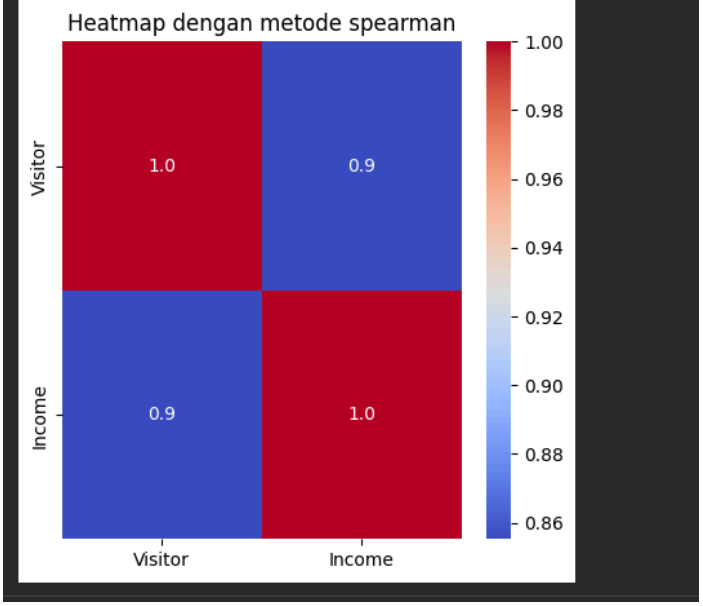
	Visitor	Income
Visitor	1.000000	0.553579
Income	0.553579	1.000000

Paling umum digunakan serta default method dari df.corr(). Digunakan untuk data normal & hubungan linear.

- **Visualisasi Heatmap**

```
plt.figure(figsize=(5,5))
plt.title('Heatmap dengan metode spearman')
sns.heatmap(corr_spearman, annot=True, cmap='coolwarm', fmt=".1f")
```

```
<Axes: title={'center': 'Heatmap dengan metode spearman'}>
```

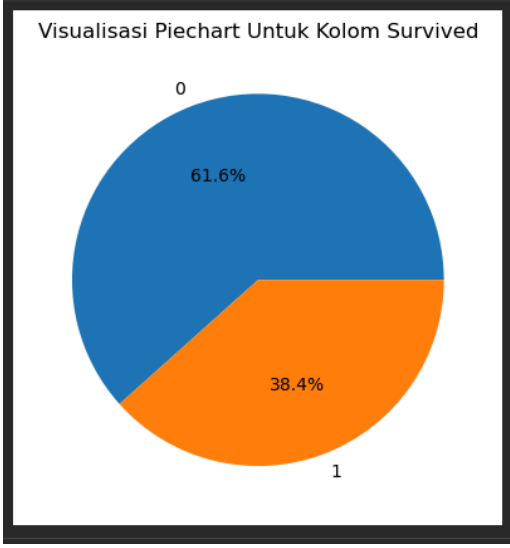


Heatmap digunakan untuk memvisualisasikan data dalam bentuk matriks, biasanya berupa angka, menggunakan variasi warna untuk merepresentasikan nilai. Fungsi ini sering digunakan untuk menampilkan korelasi antara variabel, distribusi data, atau matriks lainnya, sehingga mempermudah analisis pola atau hubungan antar data. Cara membaca heatmap :

+1 : hubungan positif sempurna
-1 : hubungan negatif sempurna
0 : tidak ada hubungan

- **Visualisasi Pie Chart**

```
plt.pie(df['Survived'].value_counts(), labels=df['Survived'].value_counts().index, autopct='%1.1f%%')
plt.title("Visualisasi Piechart Untuk Kolom Survived")
plt.show()
```



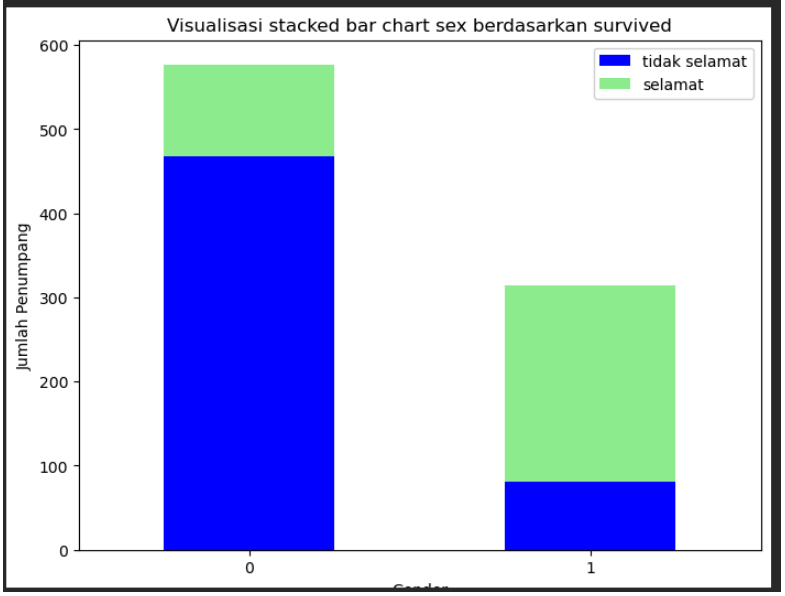
Pie Chart digunakan untuk menampilkan pembagian proporsi data kategori dalam bentuk persentase yang mudah dilihat.

- **Stacked Bar Chart**

```
survival_counts = df.groupby(["Sex", "Survived"]).size().unstack()

survival_counts.plot(kind="bar", stacked=True, color=["blue", "lightgreen"], figsize=(8, 6))

plt.xlabel("Gender")
plt.ylabel("Jumlah Penumpang")
plt.title("Visualisasi stacked bar chart sex berdasarkan survived")
plt.legend(["tidak selamat", "selamat"])
plt.xticks(rotation=0)
plt.show()
```



Stacked Bar Chart digunakan untuk melihat total suatu kategori sekaligus bagian-bagian penyusunnya dalam satu batang.

3. Modelling

- **Menentukan Independent (Fitur) & Dependent (Target)**

1. Menentukan secara langsung

```
X = df[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare',
       'Embarked', 'HasCabin']]
y = df['Survived']
```

Variabel dependen adalah variabel yang ingin kita prediksi atau analisis, biasanya merupakan target atau output. Sedangkan variabel independen adalah variabel-variabel yang digunakan untuk memprediksi variabel dependen, biasanya berupa fitur atau input. **Contoh (pada gambar): x** adalah sekumpulan variabel independen yang mencakup semua kolom kecuali kolom terakhir, sedangkan **y** adalah variabel dependen yang diambil dari kolom terakhir. Namun, pembagian ini harus dilakukan berdasarkan pemahaman masalah, bukan sekadar posisi kolom.

2. Menentukan Dengan .loc

```
X = df.loc[:, ['Embarked', 'Cabin']]
y = df.loc[:, 'Survived']
```

3. Menentukan Dengan .iloc

```
X = df.iloc[:, 0:3]
y = df.iloc[:, 3]
```

- **Feature Scalling**

Feature scaling digunakan untuk menyamakan skala nilai pada fitur-fitur dalam dataset agar tidak ada fitur yang mendominasi perhitungan karena memiliki rentang nilai yang jauh lebih besar daripada fitur lainnya. Terdapat dua cara untuk melakukan feature scaling, yaitu:

1. Normalization

$$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}}.$$

Normalization digunakan untuk mengubah nilai-nilai pada dataset menjadi berada dalam rentang tertentu, biasanya antara 0 dan 1. Normalization berguna untuk algoritma yang sensitif terhadap skala fitur, seperti **K-Nearest Neighbors (KNN)** atau **Neural Networks**, agar setiap fitur memiliki pengaruh yang seimbang dalam perhitungan.

2. Standarization

$$X_{standardized} = \frac{X - \mu}{\sigma}.$$

Standardization adalah teknik dalam feature scaling yang digunakan untuk mengubah data sehingga memiliki rata-rata (mean) 0 dan standar deviasi (standard deviation) 1. Di mana **X** adalah nilai asli, **μ** adalah rata-rata (mean), dan **σ** adalah standar deviasi dari data. Standardization digunakan untuk menyamakan skala fitur tanpa memperhatikan rentangnya, sehingga cocok untuk algoritma seperti **Linear Regression**, **Support Vector Machines (SVM)**, dan **Principal Component Analysis (PCA)**. Teknik ini memastikan data memiliki distribusi yang seragam, membuat model lebih stabil dan akurat.

- Split Data Training & Testing

1. Random Train-Test Split

```python
def split_data(X, y, train_size=0.8, random_state=42):
  np.random.seed(random_state)
  idx = np.random.permutation(len(X))
  split_ratio = int(train_size * len(X))

  X_train, X_test = X.iloc[idx[:split_ratio]], X.iloc[idx[split_ratio:]]
  y_train, y_test = y.iloc[idx[:split_ratio]], y.iloc[idx[split_ratio:]]

  return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = split_data(X, y)
```

Dalam machine learning, pembagian data menjadi **training** dan **testing** sangat penting untuk menghindari overfitting dan memastikan model dapat menggeneralisasi dengan baik pada data baru. Data **training** digunakan untuk melatih model, sedangkan data **testing** digunakan untuk mengevaluasi kinerja model setelah pelatihan.z
Pada kode ini, **random_state** diatur untuk memastikan hasil yang konsisten pada setiap eksekusi. Kemudian, **np.random.seed()** digunakan untuk mengatur seed acak agar proses pembagian data tetap sama. Selanjutnya, ukuran data untuk **train_size** ditentukan sebagai 80% dari total data. Fungsi **np.random.permutation(len(x))** untuk mengacak urutan indeks data. Dengan menggunakan indeks yang telah diacak, data dibagi menjadi **x_train** dan **x_test** untuk fitur, serta **y_train** dan **y_test** untuk target.

2. Stratified Split Data

```python
def stratified_split_data(X, y, train_size=0.8, random_state=42, frac=None):
  np.random.seed(random_state)

  if frac:
    X = X.sample(frac=frac, random_state=random_state)
    y = y.loc[X.index]

  X_train_list, X_test_list = [], []
  y_train_list, y_test_list = [], []

  for class_value in np.unique(y) :
    class_idx = np.where(y == class_value)[0]

    idx = np.random.permutation(class_idx)
    split_ratio = int(len(idx) * train_size)

    X_train_list.append(X.iloc[idx[:split_ratio]])
    X_test_list.append(X.iloc[idx[split_ratio:]])
    y_train_list.append(y.iloc[idx[:split_ratio]])
    y_test_list.append(y.iloc[idx[split_ratio:]])

  X_train = pd.concat(X_train_list).reset_index(drop=True)
  X_test = pd.concat(X_test_list).reset_index(drop=True)
  y_train = pd.concat(y_train_list).reset_index(drop=True)
  y_test = pd.concat(y_test_list).reset_index(drop=True)

  return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = stratified_split_data(X, y )
```

Teknik split data yang membagi dataset menjadi train-test dengan tetap menjaga proporsi setiap kelas agar tetap sama. Penting untuk dataset klasifikasi.

- **Simple Linear Regression (OLS statistik)**

Model ini memiliki satu fitur dan satu target.

1. **Mencari Slope / Koefisien**

$$m = \frac{n\sum(xy) - \sum x \sum y}{n\sum(x^2) - (\sum x)^2}$$

```python
slope = (len(x) * np.sum(x * y) - np.sum(x) * np.sum(y)) / (len(x) * np.sum(x**2) - np.sum(x)**2)
```

Keterangan:
x: Variabel independen.
y: Variabel dependen.
n: Panjang variabel independen.
m: Koefisien

2. **Mencari Intercept / Bias**

$$b = \frac{\sum y - m\sum x}{n}$$

```python
intercept = (np.sum(y) - slope * np.sum(x)) / len(x)
```

Keterangan:
x: Variabel independen.
y: Variabel dependen.
n: Panjang variabel independen.
m: Koefisien
b: Bias

3. **Mencari Prediksi**

$$\hat{y} = mX_{\text{test}} + b$$

```
y_pred = slope * x_test + intercept
```

Keterangan:
Xtest: Data testing variabel X.
m: Koefisien
b: Bias
^y: Nilai prediksi Y

- **Simpel Linear Regression (OLS dengan rata-rata)**
  1. **Mencari Slope / Koefisien**

$$m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

```
slope = np.sum((x - np.mean(x))*(y - np.mean(y))) / np.sum((x - np.mean(x))**2)
```

  2. **Mencari Intercept / Bias**

$$b = \bar{y} - m\bar{x}$$

```
intercept = np.mean(y) - slope * np.mean(x)
```

  3. **Mencari Prediksi**

$$\hat{y} = mX_{\text{test}} + b$$

```
y_pred = slope * x_test + intercept
```

- **Simple Linear Regression (OLS Normal Equation)**
  1. **Mencari Koefisien & Bias**

$$\beta = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

```
beta = np.linalg.inv(X_train_b.T @ X_train_b) @ X_train_b.T @ y_train
```

Keterangan:

- **X:** Matriks input (termasuk kolom 1 untuk intercept)
- **$X^\top$:** Transpose dari matriks X
- **$(X^\top X)^{-1}$:** Invers dari perkalian $X^\top X$.
- **y:** Vektor variabel target (dependent variable)
- **β:** Vektor koefisien regresi (β0,β1,β2,...,βn).

  2. **Mencari Prediksi**

$$\hat{y} = X_{b,\text{test}}\, \beta$$

```
y_pred = X_test_b @ beta
```

Berikut Keseluruhan Fungsi Linear Regression Menggunakan Normal Equation :

```python
def bias(x):
    return np.c_[np.ones(x.shape[0]), x]

def linear_regression(x_train, y_train, x_test):
    x_train_b, x_test_b = bias(x_train), bias(x_test)

    beta = np.linalg.inv(x_train_b.T @ x_train_b) @ x_train_b.T @ y_train

    y_pred = x_test_b @ beta

    return beta, y_pred

beta, y_pred = linear_regression(x_train, y_train, x_test)
```

- **Multiple Linear Regression (Menggunakan Normal Equation)**

  Model ini memiliki lebih dari satu fitur atau minimal 2 dan satu target.

- **Polynomial Linear Regression (Menggunakan Normal Equation)**

  Model ini memiliki satu fitur atau lebih dan satu target.

```python
degree = 2

# buat design matrix pakai vander
X_train = np.vander(x_train, N=degree+1, increasing=True)

# hitung beta (normal equation)
beta = np.linalg.inv(X_train.T @ X_train) @ (X_train.T @ y_train)

# prediksi
X_test = np.vander(x_test, N=degree+1, increasing=True)
y_pred = X_test @ beta
```

- **Multivariate Linear Regression (Menggunakan Normal Equation)**

  Model ini memiliki satu fitur atau lebih dan lebih dari satu target.

4. Evaluasi Model Regression

- **R-squared**

  R-squared adalah metrik dalam regresi linear yang mengukur seberapa baik model menjelaskan variasi data. Nilainya berkisar antara 0 dan 1, di mana nilai 0 menunjukkan model tidak dapat menjelaskan variasi data sama sekali, sedangkan nilai 1 menunjukkan model mampu menjelaskan seluruh variasi data dengan sempurna.

$$R^2 = 1 - \frac{RSS}{TSS}$$

```
r2 = 1 - (np.sum((y_te - y_pred)**2) / np.sum((y_te - y_te.mean())**2))
```

RSS (Residual Sum Of Squares) : Menghitung selisih kuadrat antara nilai aktual dan nilai prediksi model.

$$RSS = \sum (y_i - \hat{y}_i)^2$$

TSS (Total Sum Of Squares) : Menghitung total variasi dalam data dengan menjumlahkan kuadrat selisih antara setiap nilai data dan rata-rata data.

$$TSS = \sum (y_i - \bar{y})^2$$

- **MSE (Mean Squared Error)**

  Mean Squared Error adalah metrik evaluasi dalam regresi yang mengukur rata-rata kesalahan kuadrat antara nilai prediksi dan nilai aktual. Semakin kecil nilai MSE, semakin baik kinerja model.

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

```
mse = np.mean((y_te - y_pred)**2)
```

- **MAE (Mean Absolute Error)**

Mean Absolute Error adalah metrik evaluasi dalam regresi yang mengukur rata-rata kesalahan absolut antara nilai prediksi dan nilai aktual. MAE menunjukkan seberapa besar kesalahan rata-rata tanpa memperhitungkan arah kesalahan (positif atau negatif), sehingga lebih mudah diinterpretasikan karena memiliki satuan yang sama dengan data aslinya. Semakin kecil nilai MAE, semakin baik kinerja model dalam memprediksi data.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

```
mae = np.mean(np.abs(y_te - y_pred))
```

- **RMSE (Root Mean Squared Error)**

Root Mean Squared Error adalah metrik evaluasi regresi yang mengukur rata-rata kesalahan antara nilai prediksi dan nilai aktual, tetapi dalam skala yang sama dengan target karena merupakan akar kuadrat dari MSE.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

```
rmse = np.sqrt(mse)
```

- **Adjusted R-squared**

Adjusted R-squared adalah versi perbaikan dari R-squared yang digunakan dalam multiple linear regression, tujuannya mencegah model tampak bagus hanya karena menambah variabel.

$$R^2_{adj} = 1 - \left(\frac{(1 - R^2)(n-1)}{n-k-1}\right)$$

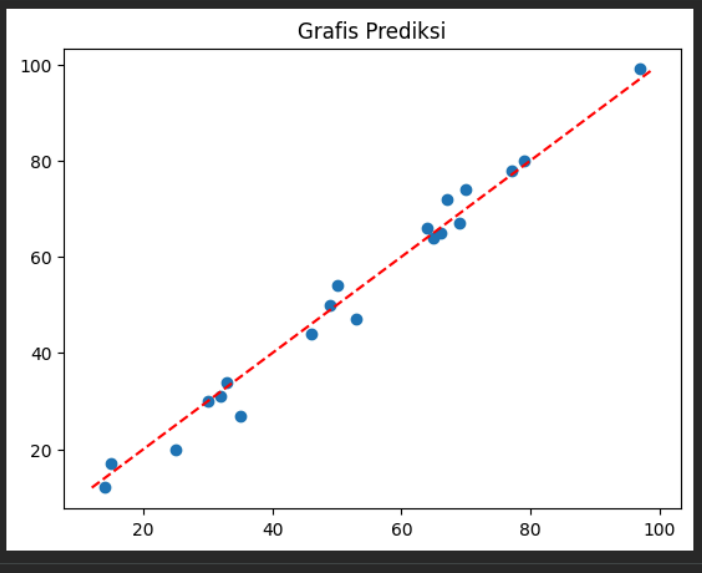
Keterangan:

- **n:** Jumlah data sample data test
- **k :** jumlah fitur

5. Visualisasi Grafik prediksi

Berikut salah satu contoh visualisasi grafik prediksi dengan algoritma simple linear regression, untuk algoritma lain bisa berbeda caranya.

```
plt.scatter(y_pred, y_te)
plt.plot([y_te.min(), y_te.max()], [y_te.min(), y_te.max()], '--r')
plt.title('Grafis Prediksi')
plt.show()
```



Istilah :

1. **Underfitting :** Terjadi ketika model terlalu sederhana, sehingga tidak mampu menangkap pola dalam data. Ciri-cirinya : R2 pada data train dan test rendah, serta tingkat error tinggi.
2. **Overfitting :** Terjadi ketika model terlalu rumit, sehingga sangat bagus di data train(menghafal) tetapi buruk pada data test. Ciri-cirinya : R2 pada data train **sangat** tinggi, sedangkan R2 pada data test turun jauh. serta Error test memburuk (Gap besar).